

claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-af709367aaddfde76.jsonl`
- SHA-256: `3bf7f9decfa6ed821df84b6c52c68000a5891d857ede8758c185556f8fcf964d`
- Source modified: `2026-07-07T04:13:37+00:00`
- Imported at: `2026-07-08T16:00:26+00:00`
- project: `wf_ddb0b3fd-b40`
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`
```

Transcript

1. user (2026-07-07T04:10:28.570Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:

- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:

- file: backend/pipeline/detectors/native_wasb_runner.py
- line: 251
- summary: The nvdec_resident healthcheck gate only verifies CUDA (torch probe) and never probes PyNvVideoCodec importability or the NVDEC driver userspace libs, so a misconfigured box passes healthcheck and fails only inside the paid inference subprocess.
- failure scenario: On a GPU box where gpu_setup.sh step [2/6] was skipped (no libnvcuvid.so.1) or PyNvVideoCodec failed to install, decode_backend=nvdec_resident passes healthcheck ('torch 2.6.0 cuda True'), trajectory_hybrid proceeds to run_predict, and wasb_infer.py dies at NvdecResidentFrontend.__init__ (wasb_infer.py:998-1007) ? the job is marked a recoverable detector Failure after subprocess launch and can retry-fail repeatedly, instead of failing at the Phase-1 gate whose stated purpose (comment at line 248-250) is to 'fail loud instead of letting wasb_infer.py die mid-run on a paid box'.
- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario

actually happens in the code as written. Default to isReal=false if the scenario cannot occur, is already guarded, or rests on a misreading.

2. user (2026-07-07T04:10:36.234Z)

```
57:# Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution probe
64:# healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-run.
65:_VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
99:    # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env only).
135:    # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
136:    # lazily probed by run_predict if healthcheck was skipped. None = not yet probed.
163:    """CUDA availability, cached across healthcheck + run_predict so we probe at most
once."""
194:    def healthcheck(self) -> Tuple[bool, str]:
228:        return False, "native torch healthcheck timed out."
251:        if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
253:            f"decode_backend=nvdec_resident requires CUDA, but the effective device is "
288:        if self.cfg.decode_backend == "nvdec_resident":
303:        if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
304:            # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on a
307:            "decode_backend=nvdec_resident requires CUDA, but the effective device is "
```

3. user (2026-07-07T04:10:37.159Z)

```
180        repo_src = self._repo_src()
181        try:
182            os.makedirs(repo_src, exist_ok=True)
183            shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
184            return True
185        except OSError as e:
186            logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
187            return False
188
189    def _rm_rf(self, path: Optional[str]) -> None:
190        """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
191        if path:
192            shutil.rmtree(path, ignore_errors=True)
193
194    def healthcheck(self) -> Tuple[bool, str]:
195        """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
196        if not self.cfg.weights_path:
197            return False, (
198                "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
199            )
200        weights = os.path.expanduser(self.cfg.weights_path)
201        if not os.path.exists(weights):
202            return False, f"WASB weights not found at {weights}"
203        repo_src = self._repo_src()
204        if not os.path.isdir(repo_src):
205            return False, (
206                f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
207                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
208            )
209        # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
210        # argparse rc=2 from wasb_infer.py at run time.
211        if self.cfg.device not in _VALID_DEVICES:
212            return False, (
213                f"unknown device {self.cfg.device!r} ? valid: {'',
''.join(_VALID_DEVICES)})."
214            )
```

```

215         if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
216             return False, (
217                 f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
218                 f"{', '.join(_VALID_DECODE_BACKENDS)}."
219             )
220         try:
221             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
222         except FileNotFoundError:
223             return (
224                 False,
225                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
226             )
227         except subprocess.TimeoutExpired:
228             return False, "native torch healthcheck timed out."
229         out = (res.stdout or "").strip()
230         if res.returncode != 0:
231             return (
232                 False,
233                 f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
234 out).strip()}",
235             )
236         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
237 subprocess).
238         self._cuda_available = "cuda True" in out
239         ctx = DeviceContext(self.cfg.device, self._cuda_available)
240         if ctx.cuda_required_but_missing:
241             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
242 a silent
243             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
244             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
245         if ctx.fell_back:
246             logger.warning(
247                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
248                 "inference will be slow. Set indexing.wasb.device=cuda to require a
249 GPU.",
250                 out,
251             )
252         # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
253 the
254 # config validator already rejects) cannot run the GPU-resident path ? fail loud
255 # instead of letting wasb_infer.py die mid-run on a paid box.
256         if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
257             return False, (
258                 f"decode_backend=nvdec_resident requires CUDA, but the effective device
259 is "
260                 f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
261             )
262         return True, out
263
264 def run_predict(
265     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
266 ) -> Optional[str]:
267     """Run WASB natively on a video. Returns the path to the trajectory CSV, or
268 None.
269
270     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
271 WSL
272 runner does, and the CSV is named ``{stem}_{vid12}_wasb.csv`` ? byte-for-byte
273 the
274 same downstream contract, so the parity comparison is apples-to-apples.
275 """
276     output_dir = os.path.abspath(output_win_dir)
277     os.makedirs(output_dir, exist_ok=True)
278     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
279     norm = str(video_win_path).replace("\\", "/")

```

```

271         stem, _ext = os.path.splitext(os.path.basename(norm))
272         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
273         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
274         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
275
276         if not self._copy_infer_script():
277             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
278             return None
279
280         weights = os.path.expanduser(self.cfg.weights_path)
281         argv = [
282             self.cfg.python_bin,
283             "wasb_infer.py",
284             "--video",
285             os.path.abspath(str(video_win_path)),
286         ]
287         frames_dir: Optional[str] = None
288         if self.cfg.decode_backend == "nvdec_resident":
289             # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
itself
290             # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
neither
291             # --stream-video nor --frames_out_dir applies and there is no frame cache to
clean.
292             argv += ["--decode-backend", self.cfg.decode_backend]
293         elif self.cfg.stream_video:
294             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
295             argv.append("--stream-video")
296         else:
297             scratch = self.cfg.scratch_dir or output_dir
298             frames_dir = os.path.join(scratch, f"{key}_frames")
299             argv += ["--frames_out_dir", frames_dir]
300             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
301             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
302             device = self._effective_device()
303             if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
304                 # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
a
305                 # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
subprocess.
306                 logger.error(
307                     "decode_backend=nvdec_resident requires CUDA, but the effective device
is "
308                     "%r ? aborting before the WASB subprocess.",
309                     device,
310                 )
311                 return None
312             argv += [
313                 "--weights",
314                 weights,
315                 "--sport",
316                 self.cfg.sport,
317                 "--device",
318                 device,
319                 "--out",
320                 out_csv,
321             ]
322             # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
323             # byte-parity guarantee) is unchanged when the knobs are unset.
324             if self.cfg.batch_size:

```

```

325         argv += ["--batch-size", str(self.cfg.batch_size)]
326     if self.cfg.prefetch_batches:
327         argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
328
329     logger.info(

```

4. user (2026-07-07T04:10:46.260Z)

```

40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63     # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
64     # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
65     _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
66
67
68     @dataclass
69     class NativeWasbConfig:
70         """Resolved settings for a Linux-native WASB run.
71
72         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
73         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
74         ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
75         """
76
77         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
78         python_bin: str = (
79             "python" # the `wasb` conda env python (invoked by path, no activate)
80         )
81         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
82         sport: str = "badminton"
83         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
84         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
85         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
86         keep_frames: bool = (
87             False # retain the decoded frame cache after success (debug only)
88         )
89         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's

```

```

90      # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
91      # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
92      stream_video: bool = True
93      # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
94      # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
95      batch_size: int = 0
96      prefetch_batches: int = 0
97      # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
98      # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
99      # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
100     decode_backend: str = "cpu"
101
102     @classmethod
103     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
104         if isinstance(idx_cfg, dict):
105             idx_cfg = IndexingConfig(**idx_cfg)
106         w = idx_cfg.wasb
107         env = os.environ.get
108         # stream_video is tri-state in config: None/absent => native default (stream);
an
109         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
110         sv = w.stream_video
111         return cls(
112             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
113             python_bin=env("WASB_PYTHON") or w.python_bin,
114             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
115             sport=w.sport,
116             device=env("WASB_DEVICE") or w.device,
117             scratch_dir=env("WASB_SCRATCH")
or env("RALLY_SCRATCH")
118             or env("TMPDIR")
119             or "",
120             timeout_sec=w.timeout_sec,
121             keep_frames=w.keep_frames,
122             stream_video=(True if sv is None else bool(sv)),
123             batch_size=int(w.batch_size or 0),
124             prefetch_batches=int(w.prefetch_batches or 0),
125             decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
126         )
127
128
129
130     class NativeWasbRunner(DetectorRunner):
131         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
132
133         def __init__(self, cfg: NativeWasbConfig):
134             self.cfg = cfg
135             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
136             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
137             self._cuda_available: Optional[bool] = None
138
139             # --- low-level native exec (the single place tests patch)
-----
140             def _run(
141                 self, argv: List[str], cwd: Optional[str] = None
142             ) -> subprocess.CompletedProcess:
143                 logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)

```

```

144         return subprocess.run(
145             argv,
146             cwd=cwd,
147             capture_output=True,
148             text=True,
149             timeout=(self.cfg.timeout_sec or None),
150         )
151
152     # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
153     def _probe_cuda(self) -> bool:
154         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
155         (missing python / timeout / import error) is treated as 'no CUDA'."""
156         try:
157             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
158         except (FileNotFoundError, subprocess.TimeoutExpired):
159             return False
160         return res.returncode == 0 and "cuda True" in (res.stdout or "")
161
162     def _cuda_is_available(self) -> bool:
163         """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
164         if self._cuda_available is None:
165             self._cuda_available = self._probe_cuda()
166         return self._cuda_available
167
168     def _effective_device(self) -> str:
169         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA

```

5. user (2026-07-07T04:10:48.040Z)

```

38:onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
42:PyNvVideoCodec env (the WSL torch-1.11 env keeps ``cpu``, its behaviour untouched).
67:# resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
980:class NvdecResidentFrontend:
983:     PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
991:         import PyNvVideoCodec as nvc
1003:             f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
1004:             f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
1005:             f"and libnvcuvid/libnvidia-encode on the box (deploy/cloudrun/Dockerfile "
1089:     fe = NvdecResidentFrontend(video_path, input_wh)
1254:         "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "

```

6. user (2026-07-07T04:10:57.709Z)

```

975         out_w //= 2
976         out_h //= 2
977     return out
978
979
980     class NvdecResidentFrontend:
981         """On-GPU frame source for the ``nvdec_resident`` detector pass (#506).
982
983         PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
984         NVDEC engine; frames arrive as CUDA tensors via DLPack and never touch the host.
985         ``read_chw`` then applies the cv2-faithful affine warp (``build_affine_grid``)
986         and ImageNet-normalizes on-device, yielding the same ``[3, inp_h, inp_w]``
987         float32 tensor per frame that the ImageDataset path assembles on the CPU.
988         """
989
990     def __init__(self, video_path: str, inp_wh, gpu_id: int = 0):
991         import PyNvVideoCodec as nvc
992         import torch

```

```

993
994     self._torch = torch
995     self._device = torch.device("cuda", gpu_id)
996     inp_w, inp_h = int(inp_wh[0]), int(inp_wh[1])
997     try:
998         self._dec = nvc.SimpleDecoder(
999             video_path, gpu_id=gpu_id, output_color_type=nvc.OutputColorType.RGB
1000         )
1001     except Exception as e:
1002         raise RuntimeError(
1003             f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
1004             f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
1005             f"and libnvcuvid/libnvidia-encode on the box (deploy/cloudrun/Dockerfile "
1006             f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
1007         ) from e
1008     self.count = len(self._dec)
1009     probe = self._decode_rgb_hwc(0)
1010     self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
1011     trans_inp_inv = _wasb_inverse_affines(
1012         self.orig_h, self.orig_w, (inp_w, inp_h), [0]
1013     )[0]
1014     self._grid = build_affine_grid(
1015         trans_inp_inv, inp_w, inp_h, self.orig_w, self.orig_h
1016     ).to(self._device)
1017     self._mean = torch.tensor(IMAGENET_MEAN, device=self._device).view(3, 1, 1)
1018     self._std = torch.tensor(IMAGENET_STD, device=self._device).view(3, 1, 1)
1019
1020     def _decode_rgb_hwc(self, index: int):
1021         """Frame ``index`` as an on-GPU HWC RGB uint8 tensor (zero-copy via DLPack)."""
1022         torch = self._torch
1023         t = torch.from_dlpack(self._dec[index])
1024         if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
1025             t = t.permute(1, 2, 0) # CHW layout variant ? HWC
1026         if t.dim() != 3 or t.shape[-1] != 3:
1027             raise RuntimeError(
1028                 f"unexpected NVDEC frame shape {tuple(t.shape)} for frame {index} ? "
1029                 f"expected 3-channel RGB (is output_color_type=RGB supported here?)"
1030             )
1031         return t
1032
1033     def read_chw(self, index: int):
1034         """Decoded frame ``index`` ? the normalized ``[3, inp_h, inp_w]`` float32 CUDA

```

7. user (2026-07-07T04:11:08.879Z)

```

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py:123:
# onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine `grid_sample`
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py:128:
# runner only ? needs the torch-2.x + PyNvVideoCodec WASB env (deploy/cloudrun/Dockerfile)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py:129:
# with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores it.
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/detectors/native_wasb_runner.py:290:
# (PyNvVideoCodec
NVDEC) ? no PNG extraction and no cv2 streaming, so neither

```

8. user (2026-07-07T04:11:11.524Z)

```

10:+ # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
15:+ # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
16:+ # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores it.
24:+ # healthcheck hard-fails if auto resolves to cpu with this backend on.
38:@@ -60,6 +60,10 @@ _CUDA_PROBE_CODE = (

```

```

43:## healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-run.
78:         res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
102:##         # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so neither
114:##         # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on a
139:## onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
143:## PyNvVideoCodec env (the WSL torch-1.11 env keeps ``cpu``, its behaviour untouched).
153:## resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
380:##     PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
388:##         import PyNvVideoCodec as nvc
400:##         f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
401:##         f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
402:##         f"and libnvcuvid/libnvidia-encode on the box (deploy/cloudrun/Dockerfile "
643:##         "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
691:## * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506 modernized
stack,
701:## the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and PyNvVideoCodec
needs
715:##     && cp -a /tmp/drv/libnvcuvid.so.* /tmp/drv/libnvidia-encode.so.* /nvlibs/ \
726:## PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
729:## RUN ln -sf "libnvcuvid.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvcuvid.so.1 \
743:## PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg build
and could
758:##     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
780:## PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
782:## PyNvVideoCodec needs (compute-only driver images ship neither).
799:## echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
800:## Compute-only driver images (GCE DLVM, DO gpu-*-base) ship neither libnvcuvid.so.1 nor
803:## if [ ! -f /usr/lib/x86_64-linux-gnu/libnvcuvid.so.1 ] || [ ! -f /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 ]; then
805:##     echo "[gpu]         extracting libnvcuvid + libnvidia-encode from driver $DRV ..."
809:##     sudo cp -a /tmp/drv-extract/libnvcuvid.so.* /tmp/drv-extract/libnvidia-encode.so.*
/usr/lib/x86_64-linux-gnu/
811:##     && sudo ln -sf "libnvcuvid.so.$DRV" libnvcuvid.so.1 \
833:## echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506
stack)"
845:## python -m pip install -q PyNvVideoCodec
869:##     import PyNvVideoCodec as pncv
870:##     print("[gpu]         PyNvVideoCodec", getattr(pncv, "__version__", "?"), "OK")
872:##     print("[gpu]         !! PyNvVideoCodec import failed:", repr(e)[:150])
887:##> | device NVIDIA L4` (plus a `PyNvVideoCodec ... OK` line ? the #506 GPU-resident decode
dep) and
901:## Ship the engine as **ONE pip-installable artifact whose behaviour is selected entirely by
the already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` /
Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? **not** a
frozen monolith, **not** an Electron/Tauri shell. The hard constraint that forces scope: torch
is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a *second,
isolated* torch env (conda py3.10 + torch 2.6+cu124 + PyNvVideoCodec since #506) shelled out via
`$WASB_PYTHON` `[verified native_wasb_runner.py:89]` ? so no single venv/freeze can deliver the
GPU-quality path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, *zero*
torch/GPU) that preserves CUJ-1..5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only**
until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config).
**LIGHT is the FIRST shippable increment, not the terminal scope ? the sole completion goal
(north star, L10) is the FULL quality path: native WASB + BYO training + in-app annotation.**
Delivery is **cross-platform (Windows/macOS/Linux): the same local webserver powers the UI on
all three, with NO native component** (L9). **Five small engine prerequisites must land before
any packaging push** ($7 P0). The single most actionable finding: **all mutable state is CWD-
relative today and `--setup`/server disagree on `config.json`** ? a silent data-loss footgun
that P0 must fix first.

```

9. user (2026-07-07T04:11:34.790Z)

```

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py:137:
# healthcheck hard-fails if auto resolves to cpu with this backend on.
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-

```

```

indexer/backend/config/presets.py:111:    treat a GPU mismatch as a WARNING (the runner
healthcheck is the authority), never a hard
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/startup.py:13:
own healthcheck is the real authority. A warning surfaces it early without false-failing.
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/config/startup.py:216:    # GPU heads-up (warning only ? the detector
healthcheck is the authority; the probe is
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/config/startup.py:225:    "has no GPU the detector
healthcheck will hard-fail ? set indexing.wasb.device="
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/config/startup.py:236:    "can be a false alarm; the
runner's DeviceContext/healthcheck is the authority.",
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py:381:    #
real GPU box) and importing it would add seconds to this probe. The detector healthcheck is the
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py:1238:    the
server stays torch-light; the detector healthcheck is the GPU authority. Extracted so the
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/base.py:11:- Provide a single place to evolve the healthcheck
/ error contract and thread
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/base.py:21:- healthcheck() -> tuple[bool, str]
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/base.py:31:2. Implement run_predict and healthcheck (return
(bool, str) tuple).
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/base.py:41:6. Add unit tests that exercise the contract
(isinstance, healthcheck shape, run
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/base.py:51:- healthcheck can just check "weights file exists
and torch importable + CUDA?".
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/base.py:57:The healthcheck message is already logged by the
hybrids on failure. The reporting
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/base.py:216:    def healthcheck(self) -> Tuple[bool, str]:
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/device.py:9:healthcheck hard-failed on a box with no GPU, so
a CPU-only Linux box (cpu-serving / a
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/stub_runner.py:84:    def healthcheck(self) -> Tuple[bool,
str]:
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py:105:    def healthcheck(self) ->
Tuple[bool, str]:
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py:123:    return False, "WSL
healthcheck timed out."
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/detectors/wasb_runner.py:85:    def healthcheck(self) -> Tuple[bool,
str]:

```

10. user (2026-07-07T04:11:35.194Z)

```

1070    cache + tracker tail as ``run()`` otherwise.
1071    """
1072    import time
1073
1074    import numpy as np
1075    import torch
1076
1077    from detectors import build_detector
1078
1079    _ensure_np_inf()
1080    if not torch.cuda.is_available():

```

```

1081         raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
1082
1083     cfg = _build_cfg(weights, sport, "cuda")
1084     frames_in = int(cfg["model"]["frames_in"])
1085     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
1086     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
1087     out_scales = list(cfg["model"]["out_scales"])
1088
1089     fe = NvdecResidentFrontend(video_path, input_wh)
1090     try:
1091         total = min(fe.count, limit) if limit else fe.count
1092         if total < frames_in:
1093             raise SystemExit(f"need >= {frames_in} frames, video has {total}")
1094         windows_total = total - frames_in + 1
1095         logger.info(
1096             f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
1097             f"{windows_total} windows"
1098         )
1099
1100         # ---- cache setup / resume decision (mirrors run(); keyed nvdec_resident) --- #
1101         if cache_dir is None:
1102             cache_dir = default_cache_dir(out_csv)
1103             os.makedirs(cache_dir, exist_ok=True)
1104             det_jsonl, _ = _cache_paths(cache_dir)
1105             cache_key = "video:" + osp.abspath(video_path)
1106             manifest = None if fresh else read_manifest(cache_dir)
1107             resume = manifest_compatible(
1108                 manifest or {},
1109                 frames_dir=cache_key,
1110                 weights=weights,
1111                 sport=sport,
1112                 frames_in=frames_in,
1113                 frames_total=total,
1114                 device="cuda",
1115                 decode_backend="nvdec_resident",
1116             )
1117             if resume:
1118                 windows_done, det_by_fid = load_and_clean_cache(cache_dir)
1119                 logger.info(
1120                     f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
1121                 )
1122             else:
1123                 if manifest is not None:
1124                     logger.info("cache incompatible with this run -> starting fresh")
1125                     reset_cache(cache_dir)
1126                     windows_done, det_by_fid = 0, defaultdict(list)
1127
1128             base_manifest = {
1129                 "version": CACHE_VERSION,

```

11. user (2026-07-07T04:11:47.008Z)

grep: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/trajectory_hybrid.py: No such file or directory

12. user (2026-07-07T04:11:53.784Z)

grep: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/*.py: No such file or directory
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/segmenters/trajectory_hybrid.py:283: ok, msg = runner.healthcheck()

13. user (2026-07-07T04:12:01.810Z)

```
260         max_window_duration = idx_cfg.max_window_duration
261         window_min_split_gap = idx_cfg.window_min_split_gap
262         window_hard_cap = idx_cfg.window_hard_cap
263         window_inactivity_end = idx_cfg.window_inactivity_end
264         inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
265         inactivity_min_density = idx_cfg.inactivity_min_density
266         inactivity_temporal_density = idx_cfg.inactivity_temporal_density
267         window_blob_fallback = idx_cfg.window_blob_fallback
268         window_blob_factor = idx_cfg.window_blob_factor
269         handoff_padding = idx_cfg.ai_handoff_padding
270         ai_max_workers = idx_cfg.ai_max_workers
271         log_candidates = idx_cfg.log_yolo_candidates
272         store_candidates = idx_cfg.store_yolo_candidates
273
274         detection_params = {
275             "detector": self.name,
276             "velocity_thresh": velocity_thresh,
277             "merge_gap": merge_gap,
278             "min_action_window_duration": min_window_duration,
279             **runner_params,
280         }
281
282         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
283         ok, msg = runner.healthcheck()
284         if not ok:
285             logger.error("%s detector environment not ready: %s", label, msg)
286             # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
287             # failed/retryable instead of writing an empty-but-"successful" result.
288             return Failure(
289                 message=f"{label} detector environment not ready: {msg}",
290                 is_recoverable=True,
291             )
292         logger.info("%s detector env OK: %s", label, msg)
293
294         # --- Phase 2: trajectory -> candidate rally windows ---
295         # Trajectory detectors process the whole video in one pass (they carry
296         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
297         t_start = time.time()
298         out_dir = os.path.join(output_base(self.config), self.family)
299         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
300         if not csv_path:
301             logger.error("%s produced no trajectory; aborting.", label)
302             # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
303             # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
304             # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305             # rather than a silent 0-segment "success" (the wf294 incident).
306             return Failure(
307                 message=(
308                     f"{label} detector produced no trajectory (timed out or aborted) ? "
309                     "detector failure, not a 0-rally video."
310                 ),
311                 is_recoverable=True,
312             )
313
314         points = runner.parse_trajectory_csv(csv_path)
315         logger.info(
316             "Parsed %d trajectory points (%d visible).",
317             len(points),
318             sum(1 for p in points if p.visible),
```

```

319         )
320
321     all_candidate_windows = runner.trajectory_to_action_windows(
322         points,
323         fps=fps,
324         frame_width=frame_width,
325         chunk_start=0.0,
326         velocity_thresh=velocity_thresh,
327         merge_gap=merge_gap,
328         min_window_duration=min_window_duration,
329         chunk_index=0,
330         max_window_duration=max_window_duration,
331         min_split_gap=window_min_split_gap,
332         window_hard_cap=window_hard_cap,
333         window_inactivity_end=window_inactivity_end,
334         inactivity_rally_thresh=inactivity_rally_thresh,
335         inactivity_min_density=inactivity_min_density,
336         inactivity_temporal_density=inactivity_temporal_density,
337         window_blob_fallback=window_blob_fallback,
338         window_blob_factor=window_blob_factor,
339     )

```

14. user (2026-07-07T04:12:49.058Z)

```

329     logger.info(
330         "Running native WASB inference (device=%s%s) on %s ...",
331         device,
332         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
333         os.path.basename(norm),
334     )
335     try:
336         res = self._run(argv, cwd=self._repo_src())
337     except FileNotFoundError:
338         logger.error(
339             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
340         )
341         return None
342     except subprocess.TimeoutExpired:
343         logger.error(
344             "WASB native inference timed out after %ss.", self.cfg.timeout_sec
345         )
346         return None
347     if res.returncode != 0:
348         logger.error(
349             "WASB native inference failed:\n%s",
350             (res.stderr or res.stdout or "")[-2000:],
351         )
352         return None
353     if not os.path.exists(out_csv):
354         logger.error(
355             "WASB native finished but expected CSV not found at %s", out_csv
356         )
357         return None
358     logger.info("WASB trajectory CSV: %s", out_csv)
359
360     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
361     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
362     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
363     if not self.cfg.keep_frames and frames_dir:
364         logger.info(
365             "Cache hygiene: removing frame cache for %s "
366             "(set indexing.wasb.keep_frames=true to retain).",

```

```
367         key,
368     )
369     self._rm_rf(frames_dir)
370     return out_csv
371
372     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
373     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
374     trajectory_to_action_windows = staticmethod(
375         TrackNetRunner.trajectory_to_action_windows
376     )
377
```

15. user (2026-07-07T04:13:37.245Z)

Structured output provided successfully